

Website Copilot

網站知識庫自動清理：LLM 產生 exclude_words

專案進度報告 · 2026.09.21

薛耀智

大綱

1. 背景與目標

- 承接上次進度 · 新站台零人工清理

2. 網頁清理 Survey

- 現有方案實測 · LLM 取得乾淨 Markdown

3. 自動產生 `exclude_words`

- 抽樣提議 · 全站驗證 · 效果

4. 端到端成果

- 兩站驗證 · 成果總結

5. 後續規劃

- 版本控制 · 多格式讀取 · 檢索機制

背景與目標

- **上次進度**：多站知識庫完成，Agent 可依網站自動路由至對應知識庫
- **核心問題**：每新增一個網站，都要人工檢視爬取結果、挑出雜訊寫進 `exclude_words`
- **本次目標**：新站台**零人工清理**，擴展網站知識庫不再受人工瓶頸限制

網頁清理 Survey

現有方案無法取代 exclude_words

方案	運作原理	實測結果
crawl4ai min_word_threshold	HTML 區塊字數不足即整塊移除	總行數 -52%，8 頁變空白
crawl4ai dynamic 閾值	依頁面文字密度動態調整區塊剪枝閾值	研究論文 PDF、資料集連結被移除
crawl4ai BM25	依查詢相關性分數保留區塊	區塊級過濾，無法處理行級雜訊
winnow-md	依區塊形狀與跨頁重複偵測模板雜訊	nculab 誤刪清單型正文；ncucsie 有效 (Token Reduction 38–64%)，但 a=0.5 清空 71 頁

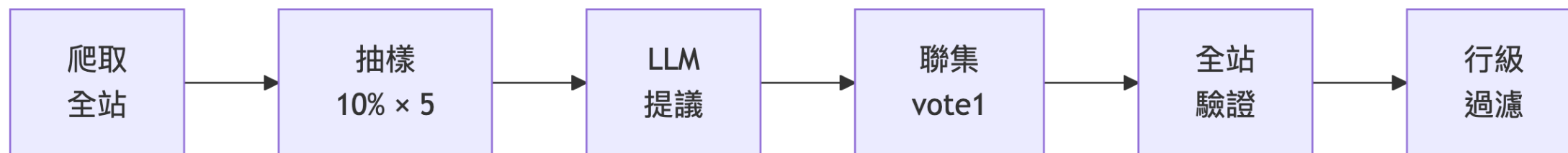
LLM 取得乾淨 Markdown 的方法

#	方法	LLM 做什麼	代表工具
1	逐頁 LLM 過濾	每頁全文重新生成乾淨 Markdown	Crawl4AI LLMContentFilter
2	專用小模型轉換	1.5B 級模型 HTML → Markdown	Jina ReaderLM-v2
3	由 LLM 產生規則 ✓	只看少數樣本頁，輸出 exclude_words	自行實作

- 方案 3：沿用既有 exclude_words 介面、成本與頁數無關、規則確定可審查
- 方案 1、2 為生成式：長頁面有改寫、截斷、靜默遺失風險

自動產生 `exclude_words`

流程



關鍵設計取舍

設計	選擇	運作原理
合併策略	各次聯集 (vote1)	5 次提議只要任一次出現的詞就保留，不要求多次一致
取樣	10% × 5 次	每次隨機抽 10% 頁面 (至少 2 頁) 交給 LLM，詞須逐字出現在至少 2 個樣本頁
驗證	低覆蓋行命中比例 > 0.1 即剔除	統計詞在全站命中的行；只出現在極少數頁面 (覆蓋率 < 5%) 的行視為正文，命中多在這類行的詞剔除

效果 (ncucsie , 204 頁)

指標	結果
與人工清單重合	99.3%
多刪行數	3 行
殘留行數	19 行
詞數 (LLM / 人工)	36 / 31
成本	約 \$0.05

端到端成果

兩站驗證結果

階段	nculab (47 頁)	ncucsie (204 頁)
爬蟲 + 自動清理	✓ 28 秒	✓ 102 秒
圖片摘要	✓ 205 秒	✓ 380 秒
RAG 建庫	✓ 78 秒	✓ 169 秒
Server 問答	✓	✓
完整流程	311 秒	651 秒

成果總結

1. **Survey**：現有方案無法取代 `exclude_words`，收斂到「LLM 產生規則」
2. **自動清理**：抽樣提議 + 全站驗證，與人工清單重合 99.3%
3. **完整流程**：單一指令跑通爬蟲 → 圖片摘要 → RAG 建庫 → Agent → Server

後續規劃

現在可以**快速擴展多個網站知識庫**，接下來著重在知識庫本身：

1. **擴增網頁讀取內容** (pdf、word、excel...)
2. **網站知識庫版本控制**
3. **加強知識庫檢索機制** (KG、Graph RAG...)